Roulette Casino en Python

Dans ce mini–projet, il vous est proposé d'implémenter en Python le jeu de la roulette que l'on retrouve dans les casinos. Plus particulièrement, on s'intéresse à la partie du jeu où l'on parie sur un *nombre*.

Le principe est le suivant : le joueur choisir un nombre entre 1 et 36 (tous deux inclus) sur lequel il mise une certaine somme d'argent. La roulette est numérotée à intervals réguliers de 0 à 36 (tous deux inclus). On fait alors tourner cette roulette et on y lance une balle. Plusieurs issues sont possibles :

— La balle tombe sur le 0 : c'est la banque qui rafle la mise, et le joueur perd.

— La balle tombe sur un nombre entre 1 et 36, mais pas sur le nombre choisir par le joueur : c'est perdu.

— La balle tombe sur le nombre choisi par le joueur : c'est gagné !

En cas de défaite, le joueur perd la mise. En cas de victoire, il remporte 36 fois la somme misée.

La version minimale de ce projet (Parties 1, 2 et 3) consistent à jouer une manche de roulette. La partie 4 propose plusieurs extensions possibles à cette version minimale. Vous pouvez également implémenter votre propre extension après discussion avec un enseignant. N'hésitez pas à demander de l'aide et amusez-vous bien !

1 Simuler un lancer de balle

Comme les intervals entre 0 et 36 sont réguliers, lancer une balle revient à choisir aléatoirement de façon uniforme un nombre entre 0 et 36 (lancer un dé à 36 faces).

Écrivez une fonction Python `simuler_lancer_balle` ne prenant aucun paramètre en entrée, et retournant un nombre aléatoire entre 0 et 36. *Indication :* pour ce faire, utiliser la fonction `randint` que vous pouvez appeler en ajoutant la ligne `from random import randint`. Cette fonction retourne un entier choisi aléatoirement de façon uniforme entre les deux arguments qui lui sont passés.

2 Déterminer l'issue d'une manche

Écrivez une fonction `issue_manche` prenant trois paramètres : `nombre_choisi` le nombre choisi par le joueur sur lequel il parie, `nombre_tombe`, le nombre qui est tombé à la roulette et `mise` la mise pariée par le joueur.

Cette fonction doit retourner le gain du joueur : celui-ci doit être négatif en cas de défaite et correspondre à la somme misée ; et positif en cas de victoire, égal à 36 fois la somme misée. Il vous est aussi demandé, en fonction d'une des 3 issues mentionnées plus haut, d'afficher un message adéquat avec `print`.

3 Jouer une manche complète avec un utilisateur

Écrivez une fonction `manche_roulette`, ne prenant aucun paramètre en entrée. Cette fonction ne retournera aucune valeur (on parle d'ailleurs de *procédure* dans ce cas !).

Cette fonction doit : (1) Accueillir le joueur avec un message de bienvenue (utiliser `print`). (2) Récupérer la mise de départ du joueur avec la fonction `input`[1], en passant en paramètre le message adéquat à afficher. Afficher ensuite la mise du joueur avec un `print`. (3) Récupérer le nombre sur lequel parie le joueur toujours en utilisant `input` avec un message à afficher. Afficher ensuite ce nombre. (4) Simuler le lancement de la balle avec la fonction `simuler_lancement_balle`, dont vous stockerez le résultat dans une variable `nombre_tombe`. Afficher ensuite cette valeur. (5) Récupérer les gains avec un appel à la fonction `issue_manche`, en précisant les valeurs des trois paramètres de cette fonction, à savoir : le nombre renseigné par le joueur, la mise pariée par le joueur, et `nombre_tombe`. (6) Actualiser la solde du joueur en fonction du gain obtenu. (7) Afficher finalement la solde du joueur après la manche.

4 Proposition d'extensions

Voici quelques propositions d'extensions à cette version minimale :

1. Compléter le code actuel afin de pouvoir jouer autant de manches que le joueur le souhaite. Dans un tel scénario, le joueur entre un solde de départ sur laquelle il peut jouer plusieurs manches. A chaque nouvelle manche, le joueur mise une partie de ce solde (pensez à interdire les dettes ...) et fais une manche de roulette. A chaque manche le solde du joueur est actualisé, et on lui demande s'il souhaite rejouer ou non. Si non, le joueur repart avec son solde, si oui, on continue de nouvelles manches. Si le solde devient nul, alors la partie s'achève sans l'avis du joueur.
2. En réalité, la roulette permet également de parier sur une couleur. Le principe de ce mode de jeu est décrit dans `https://fr.wikipedia.org/wiki/Roulette_(jeu_de_hasard)`. L'extension consiste donc de proposer au joueur soit de parier sur un nombre, soit de parier sur une couleur. Il faut alors coder la version « nombre », et permettre au joueur de choisir l'un ou l'autre des modes de jeu. Il est possible de combiner cette extension avec la première.
3. Il est intéressant lorsque l'on s'intéresse à des jeux de hasard d'estimer leur espérance, à savoir, le gain moyen auquel s'attendre en jouant un très grand nombre de manches. L'extension consiste donc non plus à faire jouer un joueur, mais à simuler un très grand nombre de manches, pour un nombre parié et une mise constants pour toutes les manches simulées. Le code doit permettre de simuler un nombre choisi de manches, et de calculer au fur et à mesure la moyenne des gains observée. Il vous faut enfin afficher à la fin le gain moyen calculé.

1. `input` est une fonction qui prend en paramètre un message à afficher, et retourne une valeur précisée par l'utilisateur au moment de l'exécution du code Python. Par exemple, si dans votre Notebook vous exécutez la ligne `age = input("Quel est votre âge ?")`, le message `"Quel est votre âge ?"` s'affichera (comme avec un `print`), et une petite boîte apparaîtra où l'on peut rentrer une valeur. Une fois renseignée, celle-ci est stockée dans la variable `age`. **Attention :** la fonction `input` est sensible à la casse, il faut donc *caster* le type de la variable récupérée. Concrètement, il s'agit d'entourer l'appel à `input` de `int()` si vous voulez un entier, par exemple : `age = int(input("Quel est votre âge ?"))`